GUÍA GENERAL

Roadmap

De User Space al Ring 0
POSIX · Kernel Linux · eBPF · Rust

Versión 1.0 · Roadmap Sistemas Linux

Resumen

Este documento organiza el recorrido completo: siete proyectos encadenados que van desde las llamadas al sistema más básicas hasta escribir un driver del kernel, instrumentar Linux con eBPF y construir un servidor en Rust. Cada proyecto tiene su propio subject. Aquí encontrarás el mapa, las reglas comunes, el entorno de trabajo y cómo convertir todo esto en un portfolio que se note en una entrevista.

CAPÍTULO I

Filosofía

Estos subjects siguen la pedagogía que ya conoces: nadie te da la solución, te dan un objetivo preciso, restricciones claras y un criterio de evaluación. El aprendizaje ocurre en el hueco entre las dos cosas, leyendo `man`, el código fuente del kernel, la documentación oficial y equivocándote mucho.

- **Lee antes de escribir.** Cada ejercicio menciona las páginas de manual relevantes. Léelas enteras, incluida la sección ERRORS: ahí está la mitad del ejercicio.
- **Explica lo que haces.** Varios ejercicios piden un `NOTES.md`. Escribirlo te obliga a entender; además, es exactamente lo que te preguntarán en una entrevista técnica.
- **Evalúate como en una corrección.** Al final de cada subject hay una hoja de evaluación. Si puedes, pide a un compañero de 42 que te corrija con ella.
- **El orden importa.** Cada proyecto usa lo aprendido en el anterior. Saltarte fases se paga después.

CAPÍTULO II

Mapa del recorrido

#	Proyecto	Fase	Conceptos clave	Tiempo orientativo
01	rawio	1 · POSIX	syscalls, file descriptors, `read`/`write` parciales, `stat`, `lseek`, directorios, `strace`	1–2 semanas
02	procs	1 · POSIX	`fork`, `execve`, `wait`, pipes, señales, `sigaction`, grupos de procesos	1–2 semanas
03	psh	1 · POSIX	shell completa: parser, pipelines, redirecciones, señales, **job control**	4–6 semanas
04	ring0	2 · Kernel	entorno con VM, primer LKM, parámetros, `/proc`, leer un oops	1 semana
05	ringdev	2 · Kernel	character device, `copy_to_user`, concurrencia, wait queues, `poll`, `ioctl`	3–4 semanas
06	pktwatch	3 · eBPF	libbpf, CO-RE, kprobes/fentry, mapas, verificador, contadores por proceso	2–3 semanas
07	rhttpd	3 · Rust	ownership, threads, parser HTTP robusto, `unsafe`-free, tests y fuzzing	3–4 semanas

Los tiempos asumen unas 15–20 horas semanales. Es un recorrido de unos 4–6 meses: realista para hacerlo en paralelo al cursus.

> **NOTA**
> La Fase 3 no es «eBPF **o** Rust»: el orden recomendado es eBPF primero (encaja directamente con lo que acabas de aprender del kernel) y Rust después. Si vas justo de tiempo, prioriza el que más encaje con las ofertas que te interesan: eBPF para cloud, observabilidad y seguridad; Rust para sistemas, infraestructura y embebido.

CAPÍTULO III

Relación con el cursus de 42

El cursus ya incluye `pipex` y `minishell`, que se solapan con la Fase 1. Estos subjects están diseñados para ir más allá de ellos, no para repetirlos:

- `procs` cubre lo de `pipex`, pero añade decodificación completa de estados de salida, señales con `sigaction` y grupos de procesos, que son la base del job control.
- `psh` exige **job control** (`&`, `Ctrl+Z`, `jobs`, `fg`, `bg`, `tcsetpgrp`) y modo no interactivo. Eso es justo lo que separa una mini-shell de una shell de verdad, y minishell no lo pide.
- Mantén repositorios separados para los proyectos del cursus y para estos. Respeta las normas de 42 sobre código compartido y reutilización.

CAPÍTULO IV

Entorno de trabajo

Fase	Dónde trabajar	Por qué
1 · POSIX	Linux nativo, VM o WSL2	Todo es user space; cualquier Linux moderno sirve.
2 · Kernel	**Solo** dentro de una VM (QEMU/KVM, VirtualBox, UTM)	Un fallo en tu módulo cuelga el sistema entero. Con snapshots vuelves a un estado limpio en segundos.
3 · eBPF	VM con kernel ≥ 5.15 y BTF activado	CO-RE necesita `/sys/kernel/btf/vmlinux`. Debian 12 y Ubuntu 22.04+ lo traen.
3 · Rust	Cualquiera con `rustup`	Toolchain estable; nada especial.

```
# Paquetes base (Debian/Ubuntu)
sudo apt install build-essential gdb valgrind strace ltrace man-db manpages-dev \
                 manpages-posix-dev git clang llvm libbpf-dev bpftool \
                 linux-headers-$(uname -r) libreadline-dev
```

CAPÍTULO V

Reglas comunes a todos los proyectos

- Cada proyecto vive en su propio repositorio Git con commits pequeños y mensajes descriptivos.
- C: compila con `cc -Wall -Wextra -Werror` y un estándar explícito (`-std=c17` más el *feature test macro* que necesites, p. ej. `-D_DEFAULT_SOURCE`).
- Makefile con las reglas `all`, `clean`, `fclean`, `re` y, cuando aplique, `asan` (build con `-fsanitize=address,undefined -g`) y `test`.
- Cero fugas de memoria y de file descriptors. Un fd abierto de más es un error igual que un `malloc` sin `free`.
- Todo error de una syscall se comprueba. Si una función puede fallar, tu código debe tener un camino para ese fallo.
- Código del kernel: estilo del kernel, verificado con `scripts/checkpatch.pl`.
- Rust: `cargo fmt`, `cargo clippy -- -D warnings` y `cargo test` en verde.

REGLA

Un programa que crashea (segfault, abort, kernel oops, panic de Rust) durante la evaluación es un 0 en ese proyecto, igual que en 42. No hay medias tintas con la robustez en programación de sistemas.

CAPÍTULO VI

Bibliografía recomendada

Recurso	Para qué fase	Comentario
The Linux Programming Interface (M. Kerrisk)	1	La referencia. Capítulos de ficheros, procesos, señales y terminales.
Advanced Programming in the UNIX Environment (Stevens, Rago)	1	Clásico; el capítulo de job control es oro para `psh`.
Páginas `man` secciones 2, 3 y 7	1, 2	`man 7 signal`, `man 7 pipe`, `man 2 open`... Son la fuente primaria.
Linux Device Drivers, 3rd ed. (gratis online)	2	Antiguo en APIs, vigente en conceptos. Contrasta siempre con el código actual.
Linux Kernel Development (R. Love)	2	Visión general del kernel: procesos, scheduling, memoria, sincronización.
docs.kernel.org y elixir.bootlin.com	2, 3	Documentación oficial y navegador del código fuente por versión.
Learning eBPF (L. Rice)	3	Introducción clara y práctica a eBPF con libbpf.
libbpf-bootstrap (GitHub)	3	Plantillas mínimas de proyectos eBPF con CO-RE.
The Rust Programming Language (gratis online)	3	El capítulo final construye un servidor web multihilo: úsalo como punto de partida, no como solución.

CAPÍTULO VII

Convertirlo en portfolio

Un repositorio de sistemas bien presentado vale más que diez proyectos web. Para cada proyecto:

- **README** con: qué hace, cómo compilarlo, una demo (asciinema o GIF), decisiones de diseño y *qué aprendiste*. Esta última sección es la que leen los entrevistadores.
- **Tests automáticos** y, cuando sea posible, integración continua con GitHub Actions (Fase 1 y Rust funcionan en CI sin problemas).
- **Un diagrama** de la arquitectura: el AST de `psh`, el ring buffer de `ringdev`, el flujo kernel → mapa → user space de `pktwatch`.
- **Un post técnico** sobre el bug más difícil que encontraste. Explicar un deadlock o una race condition que depuraste demuestra más que cualquier línea del CV.

> **PISTA**
> En entrevistas de sistemas son muy habituales preguntas como «¿qué pasa exactamente cuando escribes `ls | grep txt` en una shell?» o «¿por qué no puedes llamar a `printf` dentro de un handler de señal?». Al terminar la Fase 1 deberías poder responder ambas durante diez minutos sin parar.